Sistema Adopción. Equipo 7

Versión	2.0
Fecha	10 de abril de 2026
Equipo	Juan Daniel Barrera Holan Ⓐ Andrick Iorel Marcial Ahedo Ⓔ Edgar Salgado Gónzalez Angel Carlos Hérnandez Coronel Ⓖ Gustavo Alonso Domínguez Sánchez
Detalles	**Documento evolutivo del sistema en su Iteración 2**

Especificación de Requerimientos del Sistema
- 1. Introducción
  - 1.1. Propósito del documento
  - 1.2. Alcance
    - Alcance general
    - Alcance de la Versión 2.0
  - 1.3. Definiciones relevantes
- 2. Actores del sistema
- 3. Requerimientos funcionales
- 4. Casos de Uso
  - 4.1. Casos de uso generales
  - 4.2. Evolución de casos de uso a la versión 2.0
- 5. Requerimientos No Funcionales
- 6. Modelado de datos
  - Modelo Entidad-Relación
  - Modelo Relacional

Limitaciones Actuales
- 1. Tecnologías Utilizadas
  - 1.1. Backend
  - 1.2. Frontend
  - 1.3. Complementos
- 2. Arquitectura General
  - 2.1. Cliente (Frontend)
  - 2.1. Servidor (Backend)
- 3. Flujo de Autenticación
- 4. Modelo Conceptual de Datos
  - Atributos Principales y Propósito
  - Consideraciones de Seguridad

Especificación de Requerimientos del Sistema

1. Introducción

1.1. Propósito del documento

El propósito de este documento es definir de manera precisa y ordenada los requerimientos del Sistema de Adopción de Mascotas, con el fin de establecer una base para su análisis, desarrollo y validación.

Estos lineamientos nos van a permitir identificar las funciones principales del sistema, los actores que van a interactuar con él y las condiciones generales que deben cumplirse para satisfacer las necesidades del proyecto. Además, servirá como referencia para el equipo de trabajo durante el desarrollo de la plataforma.

Para este documento en particular, también nos enfocaremos en los requerimientos y funcionalidades implementadas a su estado en la versión 2.0:

- En el apartado de *Especificación de Requerimientos del Sistema*, haremos un especial enfásis enfocado en las limitantes para esta versión, únicamente en las siguientes secciones:
    - *1.2. Alcance*
    - *4.1. Casos de uso*
- Profundizaremos completamente sobre el estado real del sistema en su versión 2.0 en el apartado de *Limitaciones Actuales.*

1.2. Alcance

Alcance general

El sistema completo consiste en una plataforma orientada a la publicación y consulta de mascotas en adopción, enfocada principalmente en perros y gatos.

A través de esta plataforma, los usuarios registrados podrán crear publicaciones de mascotas disponibles para adopción, así como consultar las publicaciones existentes dentro del sistema. De igual manera, los usuarios podrán manifestar su interés en adoptar una mascota mediante un mecanismo de contacto controlado dentro de la plataforma.

El sistema buscará facilitar el proceso de adopción de manera segura, evitando la exposición directa de información sensible de los usuarios, como correos electrónicos o direcciones exactas.

Alcance de la Versión 2.0

Para esta versión 2.0, el sistema evoluciona de una API de backend a una plataforma integral con una interfaz gráfica para el usuario (frontend) completamente funcional. El alcance se sigue mantieniendo centrado en la gestión segura del ciclo de vida de la entidad `Usuario`, aunque ahora, con interacción visual permitiendo así un flujo completo de extremo a extremo (end-to-end).

A través de la nueva interfaz web, el sistema es capaz de realizar las siguientes operaciones integradas:

- **Registro de usuarios**: Creación de cuentas mediante un formulario visual.
- **Inicio de sesión**: Autenticación segura y generación de sesión.
- **Consulta de perfil**: Visualización de la información del usuario autenticado.
- **Actualización de información**: Interfaz para modificar los datos personales del usuario autenticado (nombre, correo y código posta).
- **Cierre de sesión**: Terminación segura de la sesión activa y redirección al portal de acceso.

Todas estas funciones se comunican internamente de manera segura, garantizando la protección de credenciales y evitando la exposición directa de información sensible a usuarios no autenticados.

1.3. Definiciones relevantes

- **ERS:** Especificación de Requerimientos del Sistema. Documento que describe las funciones, restricciones y comportamiento esperado del sistema.
- **RF:** Requerimiento Funcional. Define una función o servicio que el sistema debe proporcionar.
- **RNF:** Requerimiento No Funcional. Define restricciones o atributos de calidad del sistema, como seguridad, rendimiento o disponibilidad.

- **CU:** Caso de Uso. Describe la interacción entre un actor y el sistema para alcanzar un objetivo específico.

2. Actores del sistema

- **Actor: Usuario registrado**

Descripción: Aquella persona que ha realizado el proceso de registro dentro de la plataforma y ahora puede interactuar con todas las funcionalidades dentro de esta, este actor engloba de manera general a aquellos usuarios registrados que están interesado o interactúan con una funcionalidad especifica de la plataforma, ya sean los usuarios que publican animales en adopción, o los usuarios interesados en las publicaciones de animales en adopción.

- **Actor: Publicador de mascota**

Descripción: Usuario registrado que crea una publicación de una mascota en adopción dentro de la plataforma. Su función principal es proporcionar la información del animal y recibir notificaciones cuando otro usuario manifiesta interés en la publicación.

- **Actor: Usuario no autenticado (Visitante)**

Descripción: Es cualquier persona que accede a la plataforma web sin haber iniciado sesión. Su interacción con el sistema está altamente limitada. Solo es capaz de visualizar la página para registrar una cuenta nueva y acceder al formulario de inicio de sesión. El sistema le bloquea cualquier intento de consulta del catálogo o publicación de mascotas.

- **Actor: API Externa (The Dog API / The Cat API)**

Descripción: Actor externo que provee servicios web utilizados por el sistema para consultar el catálogo de razas de perros y gatos, así como obtener información adicional sobre sus características, cuidados y temperamento. El sistema consume estos servicios para enriquecer la información mostrada en las publicaciones de mascotas.

- **Actor: Sistema de correo electrónico**

Descripción:

Actor externo encargado del envío automatizado de notificaciones por correo electrónico generadas por la plataforma. Este servicio permite informar al usuario publicador cuando otro usuario manifiesta interés en una mascota

publicada, actuando como intermediario en la comunicación inicial entre usuarios.

3. Requerimientos funcionales

RF1: Registro de usuarios en la plataforma.

Descripción:

Cualquier persona interesada en interactuar con la plataforma deberá de realizar un proceso de registro para poder acceder a las funcionalidades del sistema. Cada persona interesada en registrarse debe proporcionar de manera mínima su nombre completo, una dirección de correo electrónico válida, además de su código postal.

Criterios de Aceptación:

- Se debe asegurar que antes de dar de alta en la plataforma a un usuario, este nos haya proporcionado su nombre completo, correo electrónico válido (importante al ser el medio de contacto con los usuarios) y su código postal.
- Si es proporcionada una contraseña para autenticar el ingreso de un usuario a la plataforma, es necesario cifrar está información antes de registrarla en nuestro sistema de almacenamiento (Alta relación con los requerimientos no funcionales de seguridad de este proyecto)

RF2: Publicación de animales en adopción.

Descripción:

Cualquier usuario registrado tiene la capacidad de publicar una publicación de un animal (perro o gato) en adopción dentro de la plataforma. Cada usuario registrado interesado en está funcionalidad deberá de proporcionar el nombre del animal, una descripción física breve del animal, submitir al menos una fotografía del animal en cuestión, además del tipo (perro o gato), su raza o similar, además del código postal donde reside la mascota para poder realizar la publicación.

Criterios de Aceptación:

- Para poder realizar la publicación en la plataforma, es necesario proporcionar **al menos** la siguiente información: Nombre, descripción física, fotografías, tipo, raza y código postal del animal para realizar la publicación de adopción.

- Al deber contar con una descripción general del animal para realizar una publicación, se debe asegurar que la descripción tenga un tamaño razonable para ser almacenado y ser lo suficientemente descriptivo sin ser tan grande como para consumir más recursos de almacenamiento.

RF3: Consulta de publicaciones de mascotas disponibles

Descripción:

El sistema debe permitir que el usuario registrado consulte las publicaciones de mascotas disponibles para adopción, mostrando la información principal de cada una, como nombre, tipo, raza, descripción, fotografía y código postal.

Criterios de Aceptación:

- El sistema debe mostrar las publicaciones activas de mascotas disponibles para adopción.
- Cada publicación debe incluir la información principal de la mascota.
- El usuario debe poder visualizar la información sin necesidad de modificar la publicación.
- Solo deben mostrarse publicaciones que se encuentren disponibles.

RF4: Manifestación de interés en una publicación

Descripción:

El sistema debe permitir que el usuario registrado manifieste su interés en adoptar una mascota mediante la opción “Me interesa” dentro de la publicación. Al realizar esta acción, el sistema deberá registrar la solicitud y generar una notificación para el usuario que publicó la mascota.

Criterios de Aceptación:

- El usuario debe haber iniciado sesión para usar la opción “Me interesa”.
- El sistema debe registrar la solicitud de interés asociada a la publicación seleccionada.
- El sistema debe notificar al publicador de la mascota que existe un usuario interesado.
- Si la publicación ya no está disponible, el sistema debe impedir la solicitud y mostrar un mensaje informativo.

RF5: Visualización geoespacial segura de mascotas

Descripción:

El sistema deberá proveer una interfaz de mapa interactivo, en el que muestre a los usuarios registrados las mascotas disponibles, agrupándolas exclusivamente por su código postal para proteger su ubicación exacta.

Criterios de Aceptación:

- El mapa deberá mostrar un marcador en aquellos códigos postales que cuenten con al menos un registro de mascota y que esté disponible para adopción.
- Los marcadores en el mapa deberán corresponder a un punto central aproximado del código postal correspondiente (un sólo marcador por cada código postal).
- Al seleccionar un marcador, el sistema deberá mostrar la lista de mascotas disponibles (junto con el enlace a su información) ubicadas en ese código postal.
- El usuario podrá navegar manualmente arrastrando el mapa.

RF6: Control de acceso al sistema

Descripción:

El sistema deberá restringir el acceso a las funciones de consulta y publicación de mascotas solo a usuarios registrados, exigiendo su autenticación previa para el uso de la plataforma.

Criterios de Aceptación:

- Todo usuario no autenticado podrá acceder a la página de autenticación (funciones de registro / inicio de sesión).
- Todo usuario autenticado será redirigido a la página principal (funciones de consulta y publicación habilitadas) si intenta acceder a la página de autenticación.
- Cuando un usuario no autenticado intenta acceder a una ruta protegida (como el de consulta o publicación), el sistema denegará su acceso y lo redirigirá automaticamente a la página de autenticación con un mensaje indicando que la autenticación es requerida.
- Una vez que el usuario ingrese credenciales válidas, el sistema deberá otorgarle acceso inmediato y redirigirlo a la página principal (con las funciones de consulta y publicación).

- Si el usuario ingresa credenciales inválidas, permanece en la página de autenticación con el formulario limpio y un mensaje de credenciales erróneas.

RF7: Integración con API externa para selección y detalle de raza

Descripción:

El sistema deberá integrar las APIs públicas de "The Dog API" y "The Cat API" para enriquecer la información de las mascotas.

Criterios de Aceptación:

- Al momento de registrar un animal, el sistema deberá permitir seleccionar la raza a partir de las opciones disponibles en la API correspondiente.
- En la ficha informativa del animal, el sistema deberá mostrar una sección adicional con datos obtenidos de la API.
- Esta sección adicional deberá contener información general de la raza, recomendaciones de cuidado y características relevantes como el temperamento o nivel de actividad.

RF8: Notificación automatizada y segura por correo electrónico

Descripción:

El sistema deberá actuar como intermediario en el primer contacto, enviando automáticamente un correo electrónico al publicador cuando alguien pulse el botón "Me interesa".

Criterios de Aceptación:

- El correo generado por el sistema deberá incluir obligatoriamente al menos un dato de contacto del usuario interesado, como su nombre y su correo electrónico.
- Bajo ninguna circunstancia el sistema mostrará directamente los correos electrónicos de los usuarios en la interfaz gráfica web.

4. Casos de Uso

4.1. Casos de uso generales

Se listan los casos de uso originales planteados en el primer documento de *Especificación de requerimiento para sistema de adopción de mascotas*,

mismos que consideran el alcance general del proyecto:

- **Publicación de un animal en adopción dentro de la plataforma.**
- **Mostrar interés en adoptar una mascota**
- **Autenticación de usuarios y acceso a funcionalidades**
- **Visualización de información extendida de la raza mediante API externa**

No es necesario detallar estos casos de uso debido a este documento evoluciona a la par del estado del proyecto, por lo que nuestra intención es tomarlos como base para plantear iterativamente los nuevos casos de uso conforme la evolución del proyecto lo vaya requieriendo.

4.2. Evolución de casos de uso a la versión 2.0

Para este alcance en particular, a partir del caso *"Autenticación de usuarios y acceso a funcionalidades"*, se plantearán los siguientes casos de uso reproducibles en el estado actual del sistema, mismos que se desean mantener para versiones posteriores hasta el despliegue:

> **CU1: Protección de rutas y registro de usuarios**

Actores: Usuario no autenticado

Precondiciones:

- El usuario no cuenta con una sesión activa.

Flujo Principal:

- El usuario intenta accede a cualquier url protegida de la plataforma (como `/home` o `/update`).
- El sistema detecta que el usuario no cuenta con una sesión activa.
- El usuario no podrá acceder y el sistema lo redirige a la página de `/login`.
- El usuario decide crear una cuenta e ingresa a a la página de `/register`.
- El usuario rellena el formulario con datos válidos. Para esta versión la validación es exitosa si:
  - Llena todos los campos (nombre, correo, codigo y contraseña).
  - El correo ingresado no se encuentra ya asociada a otra cuenta en la base de datos.
- El sistema busca en la base de datos si el correo no ha sido ocupado.

- Si el correo es válido, el sistema guarda los datos del usuario con la contraseña hasheada.
- El sistema mostrará un mensaje de éxito y redirige al usuario a la página de `/login`.

Flujo Alternativo:

Si los datos son inválidos, el sistema no los guardará y le informará al usuario el motivo de invalidación como llenar los campos o error del sistema (como correo ocupado).

Postcondición:

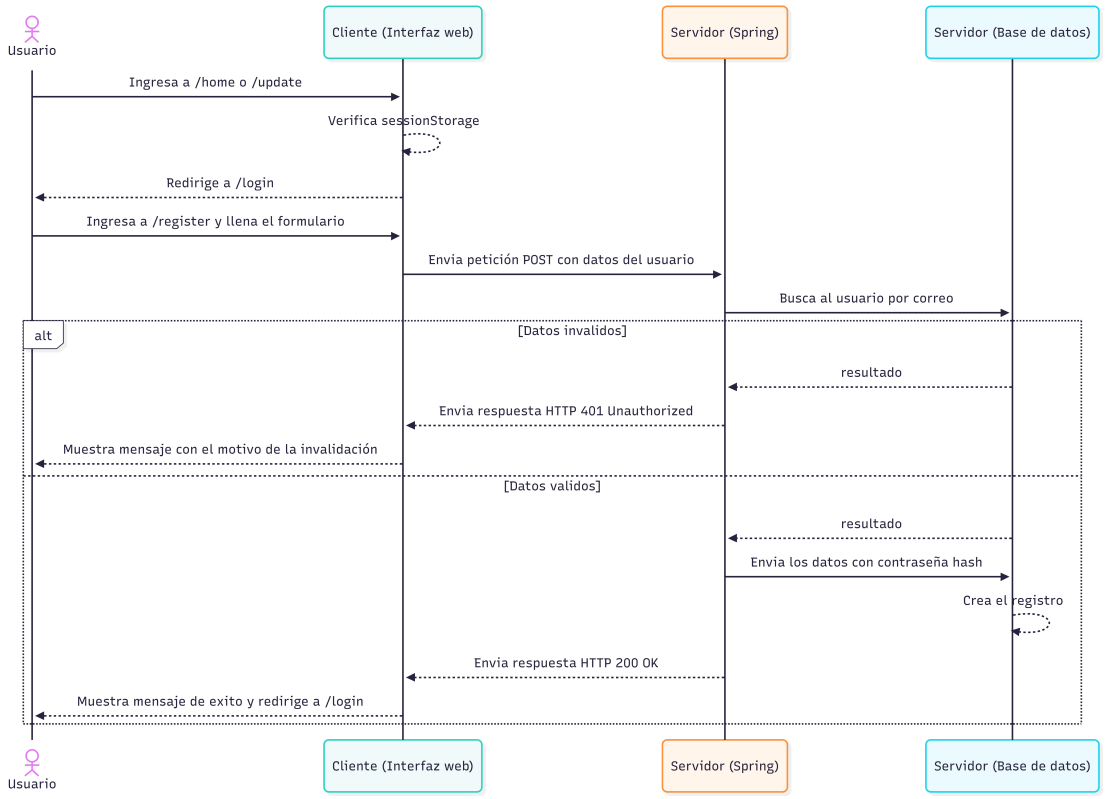
El usuario ha creado una nueva cuenta en el sistema para poder acceder.

Diagrama de secuencia:

```
sequenceDiagram
    actor Usuario as Usuario
    participant UI as Cliente (Interfaz web)
    participant API as Servidor (Spring)
    participant DB as Servidor  (Base de datos)

    Usuario->>UI : Ingresa a /home o /update
    UI-->>UI : Verifica sessionStorage
    UI-->>Usuario : Redirige a /login
    Usuario->>UI : Ingresa a /register y llena el formulari
o
    UI->>API : Envia petición POST con datos del usuario
    API->>DB : Busca al usuario por correo
    alt Datos invalidos
        DB-->>API : resultado
        API-->>UI : Envia respuesta HTTP 401 Unauthorized
        UI-->>Usuario : Muestra mensaje con el motivo de la
invalidación
    else Datos validos
        DB-->>API : resultado
        API->>DB : Envia los datos con contraseña hash
        DB-->>DB  : Crea el registro
        API-->>UI : Envia respuesta HTTP 200 OK
        UI-->>Usuario : Muestra mensaje de exito y redirige
```

```
a /login
    end
```



CU2: Inicio de sesión y consulta de información del usuario

Actores: Usuario registrado

Precondiciones:

- El usuario cuenta con un registro previo válido en la base de datos.

Flujo Principal:

- El usuario ingresa a la página de `/login` y llena el formulario con su correo electrónico y contraseña.
- El sistema captura los datos y busca el correo en la base de datos.
- Si coinciden las credenciales, el sistema genera un token de sesión y lo guarda en la base de datos asociado a ese usuario.

- El sistema crea para el usuario una sesión válida con el token y lo redirige a `/home`.
- Al cargar `/home`, el sistema verifica que exista una sesión activa, y recupera los datos mediante el token.
- El sistema muestra visualmente los datos del usuario (nombre, correo, codigo postal y ID).

Flujo Alternativo:

- Si las credenciales enviadas en el login son incorrectas, el sistema rechaza la petición y muestra el mensaje "Usuario o contraseña incorrectos".
- Si al consultar `/home` el token no existe o expiró, el sistema rechaza la petición y redirige al usuario a `/login`.

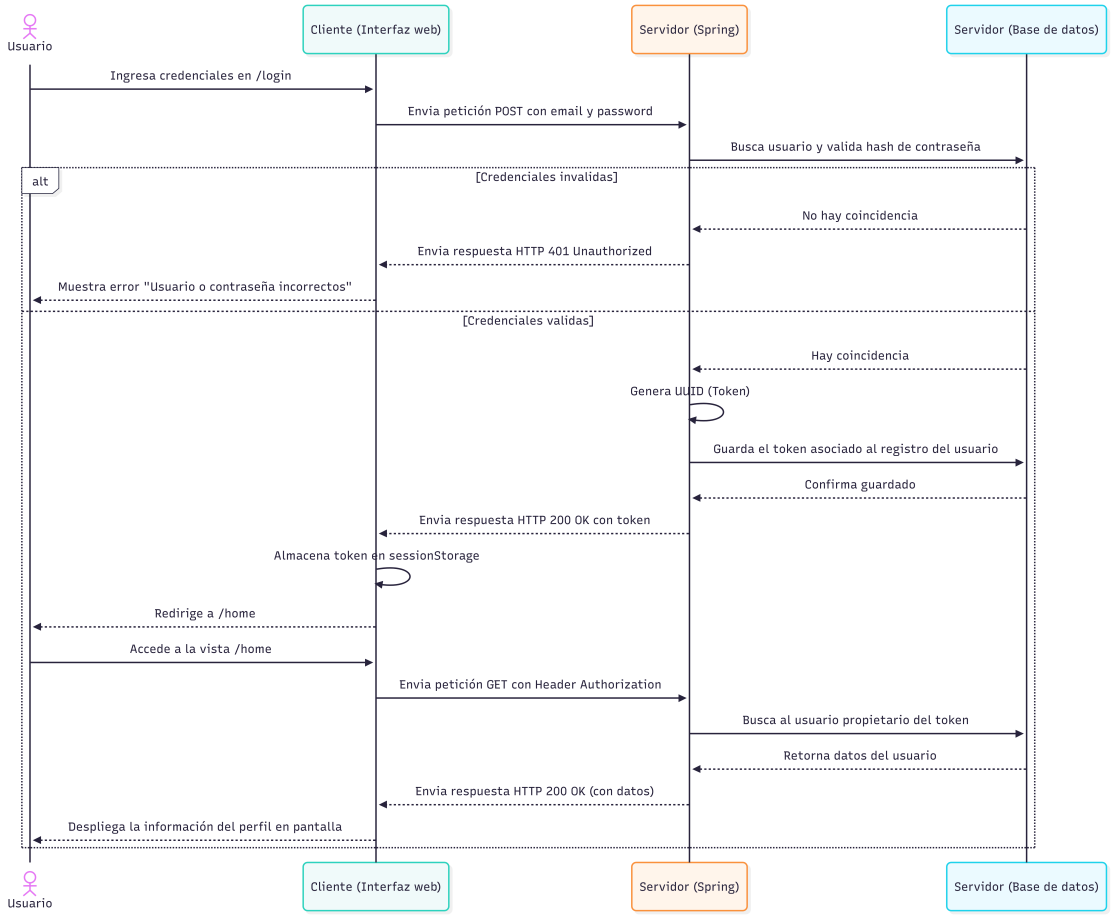
Postcondición:

El usuario tiene una sesión activa y visualiza su información privada en el panel principal.

Diagrama de secuencia:

```
sequenceDiagram
actor Usuario as Usuario
participant UI as Cliente (Interfaz web)
participant API as Servidor (Spring)
participant DB as Servidor (Base de datos)
Usuario->>UI : Ingresa credenciales en /login
UI->>API : Envia petición POST con email y password
API->>DB : Busca usuario y valida hash de contraseña
alt Credenciales invalidas
    DB-->>API : No hay coincidencia
    API-->>UI : Envia respuesta HTTP 401 Unauthorized
    UI-->>Usuario : Muestra error "Usuario o contraseña inc
orrectos"
else Credenciales validas
    DB-->>API : Hay coincidencia
    API->>API : Genera UUID (Token)
    API->>DB : Guarda el token asociado al registro del usu
ario
    DB-->>API : Confirma guardado
```

```
    API-->>UI : Envia respuesta HTTP 200 OK con token
    UI->>UI : Almacena token en sessionStorage
    UI-->>Usuario : Redirige a /home
    Usuario->>UI : Accede a la vista /home
    UI->>API : Envia petición GET con Header Authorization
    API->>DB : Busca al usuario propietario del token
    DB-->>API : Retorna datos del usuario
    API-->>UI : Envia respuesta HTTP 200 OK (con datos)
    UI-->>Usuario : Despliega la información del perfil en
pantalla
end
```



CU3: Actualización de información del usuario

Actores: Usuario autenticado

Precondiciones:

- El usuario tiene una sesión activa (token válido) y se encuentra en la vista `/update`.

Flujo Principal:

- El usuario modifica su nombre, correo electrónico, código postal y/o ingresa una nueva contraseña, y hace clic en el botón de Guardar.
- El sistema valida que los campos obligatorios no estén vacíos.
- El sistema procesa los nuevos datos y busca al usuario en la base de datos mediante el token para garantizar la identidad.
- El sistema verifica que, si el usuario cambió su correo, el nuevo correo no esté ya asociado a otra cuenta en el sistema.
- Al validar que los datos son correctos y el correo está disponible, el servidor actualiza los atributos en la base de datos.
- El sistema le muestra al usuario un mensaje de éxito, actualiza los datos de la sesión activa y lo redirige a `/home`, mostrando los nuevos datos actualizados.

Flujo Alternativo:

- Si el usuario deja campos obligatorios vacíos, el sistema detiene la petición y muestra un mensaje para rellenar todos los campos.
- Si el servidor detecta que el nuevo correo ya pertenece a alguien más, responde con un HTTP 409 Conflict y el cliente notifica el error de conflicto en pantalla.
- Si el token es inválido, aborta la operación y redirige a `/login`.

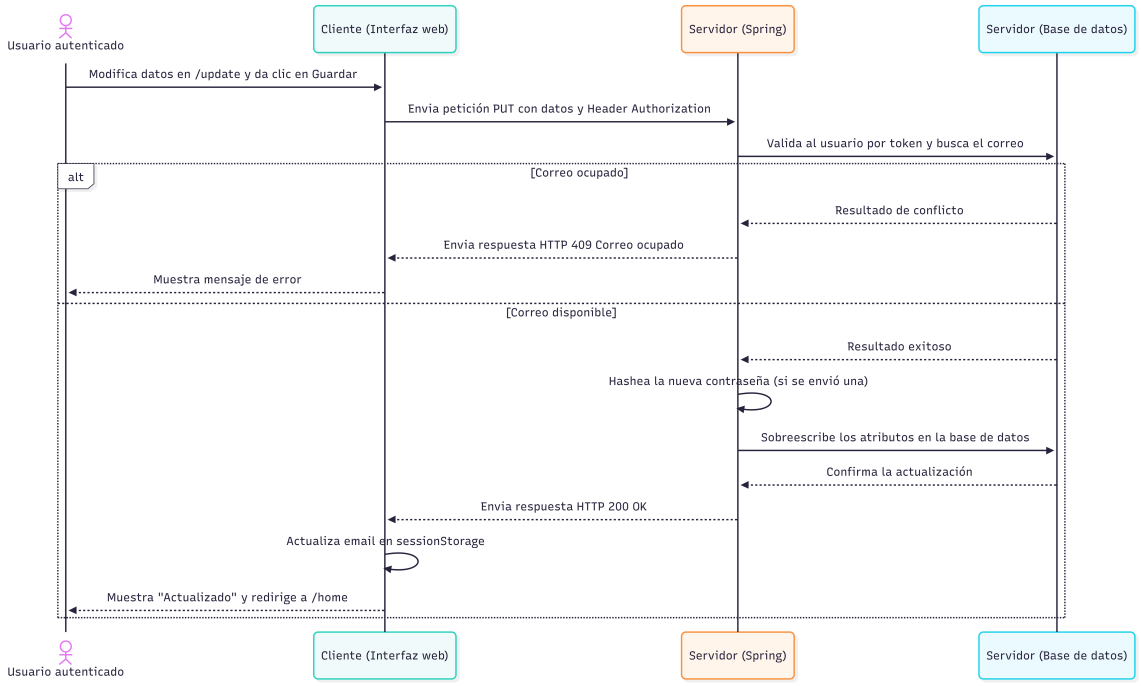
Postcondición:

La información personal del usuario se ha sobreescrito de forma persistente y segura en la base de datos.

Diagrama de secuencia:

```
sequenceDiagram
actor Usuario as Usuario autenticado
participant UI as Cliente (Interfaz web)
participant API as Servidor (Spring)
```

```
participant DB as Servidor (Base de datos)
Usuario->>UI : Modifica datos en /update y da clic en Guard
ar
UI->>API : Envia petición PUT con datos y Header Authorizat
ion
API->>DB : Valida al usuario por token y busca el correo
alt Correo ocupado
    DB-->>API : Resultado de conflicto
    API-->>UI : Envia respuesta HTTP 409 Correo ocupado
    UI-->>Usuario : Muestra mensaje de error
else Correo disponible
    DB-->>API : Resultado exitoso
    API->>API : Hashea la nueva contraseña (si se envió un
a)
    API->>DB : Sobreescribe los atributos en la base de dat
os
    DB-->>API : Confirma la actualización
    API-->>UI : Envia respuesta HTTP 200 OK
    UI->>UI : Actualiza email en sessionStorage
    UI-->>Usuario : Muestra "Actualizado" y redirige a /hom
e
end
```



5. Requerimientos No Funcionales

RNF1: Seguridad (Protección de credenciales)

Las contraseñas de los usuarios deberán almacenarse utilizando un mecanismo de hash seguro y nunca deberán guardarse en texto plano dentro del sistema de almacenamiento. Este requerimiento asegura la protección de las credenciales de los usuarios en caso de acceso no autorizado a la base de datos.

RNF2 : Privacidad de la información

El sistema deberá proteger la información personal de los usuarios, evitando la exposición directa de datos sensibles como direcciones exactas o correos electrónicos durante la interacción entre usuarios. La ubicación de los animales deberá mostrarse únicamente mediante referencias aproximadas basadas en el código postal.

RNF3 : Rendimiento del sistema

La carga de las publicaciones de mascotas disponibles, así como su visualización en el mapa interactivo, deberá completarse en un tiempo promedio no mayor a 3 segundos bajo condiciones normales de uso. Este requerimiento busca garantizar una experiencia de navegación fluida para los usuarios de la plataforma.

RNF4 : Usabilidad

La interfaz de la plataforma deberá permitir que un usuario registrado pueda navegar por las publicaciones de mascotas, consultar la información de cada animal y manifestar su interés en adopción de forma intuitiva y comprensible, sin necesidad de capacitación previa.

RNF5 : Disponibilidad del sistema

La plataforma deberá mantenerse disponible al menos el 95% del tiempo durante su periodo de operación, permitiendo el acceso continuo a las funciones principales del sistema.

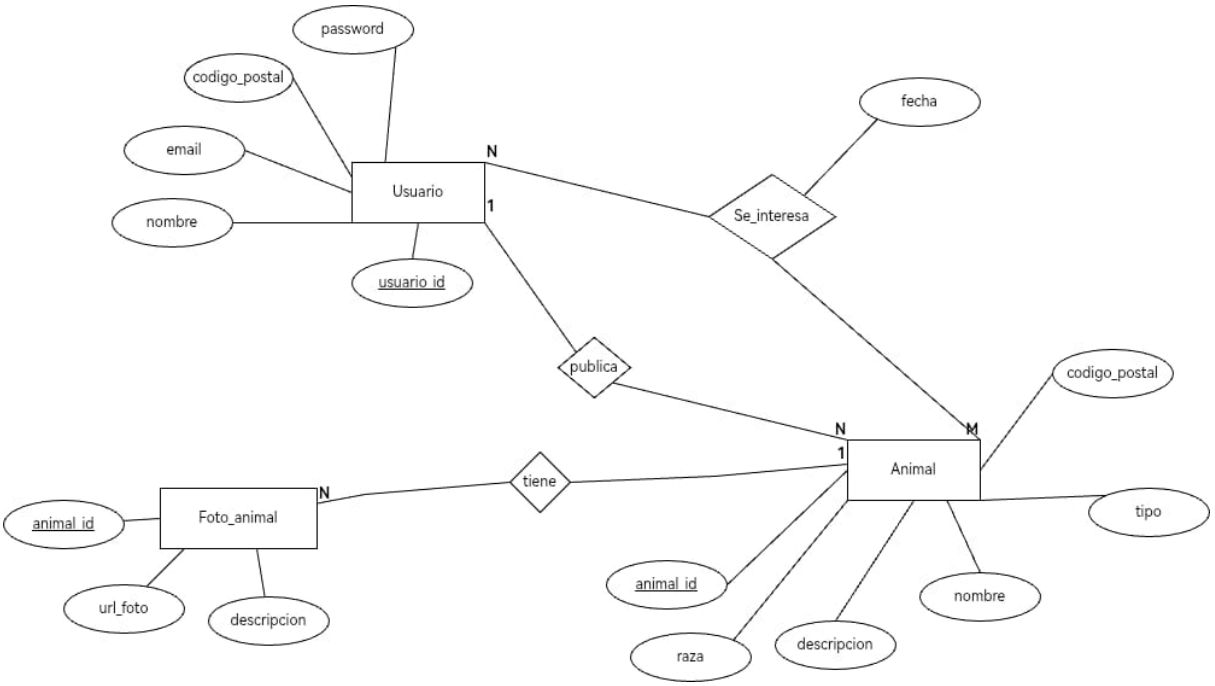
RNF6 : Integridad de los datos

El sistema deberá garantizar la consistencia e integridad de la información almacenada, evitando registros incompletos o inconsistentes en las publicaciones de animales y en las solicitudes de interés generadas por los usuarios.

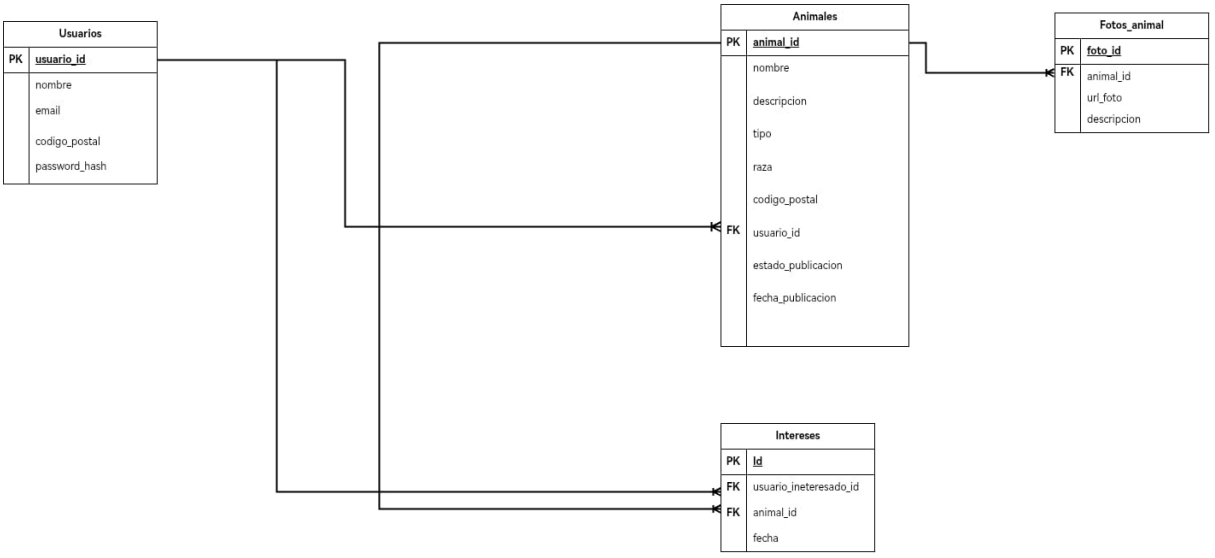
6. Modelado de datos

A la fecha de la entrega, hemos realizado está modelado de datos para conseguir los objetivos de este proyecto. Estos modelos están sujetos a cambios conforme el desarrollo de este trabajo avance.

Modelo Entidad-Relación



Modelo Relacional



Limitaciones Actuales

1. Tecnologías Utilizadas

1.1. Backend

- **Lenguaje de Programación**

> **Kotlin**

Al momento de trabajar en está iteración, el lenguaje de programación principal ha sido Kotlin, tanto para trabajar en el backend del sistema de forma general. Este lenguaje ha demostrado ser una herramienta muy poderosa y útil para la realización de este proyecto al poder acoplarse perfectamente a las necesidades de desarrollo, sobre todo en la implementación de la arquitectura por capas, permitiendo utilizar diversas funciones de Servicios y Repositorio como propiedades de un objeto, como es el caso de la creación de una instancia de UsuarioService en la clase de Controllers. No utilizamos kotlin exclusivamente, ya que se emplean bibliotecas de otros lenguajes para facilitarnos el trabajo, tales como Java; siendo otra ventaja de Kotlin el fácil acoplamiento de otros lenguajes en el proyecto.

- **Framework**

> **Spring**

Para la realización de este proyecto se emplea Spring framework, esto debido a su orientación al diseño de programas y la facilidad que otorga para el manejo de datos, esto es uno de los aspecto clave en esta parte tan temprana del desarrollo al tener que diseñar el registro y autenticación de usuarios.

- **Base de datos**

> **PostgreSQL**

De manera inicial, se ha decidido utilizar el sistema manejador de base de datos (SMBD) PostgreSQL al ser puesto a discusión en nuestro equipo. El motivo de está decisión fue lo familiarizado que está el equipo con esta herramienta y sus operaciones, lo que ha facilitado la implementación en el sistema sin la necesidad de un periodo de aprendizaje que pudiera entorpecer el desarrollo.

- **Herramientas relevantes utilizadas en el desarrollo**

Además de las herramientas previamente mencionadas, se han empleado bibliotecas de otros lenguajes, principalmente de Java, las cuáles proveen elementos básicos de seguridad que se implementaron en el almacenamiento de contraseñas de usuario, se empleó específicamente MessageDigest para hacer un hash básico de las contraseñas antes de almacenarlas. Además de SLF4J, una facade de registros simple que usamos en nuestra aplicación para llevar las operaciones hasta el momento sencillas de nuestro sistema.

1.2. Frontend

- **Lenguaje de Programación**

> **HTML5**

Se utiliza como el lenguaje de marcado principial para definir la estructura semántica de todas las vistas de la aplicación (tales como las pantallas de inicio de sesión, registro y panel de perfil). Su uso garantiza que los elementos interactivos como contenedores, formularios y botones, estén organizados lógicamente, facilitando la accesibilidad y sirviendo como el esqueleto sobre el cual los scripts manipulan el Document Object Model (DOM) de manera estructurada.

> El **DOM** (Modelo de Objetos del Documento) es una interfaz de programación que los navegadores web crean automáticamente cuando cargan un documento HTML.Su propósito principal es permitir que lenguajes de programación, como JavaScript, puedan conectarse a la página para leer o modificar su estructura, estilo y contenido en tiempo real sin necesidad de recargar el navegador.

> **CSS3**

Constituye la capa de diseño y presentación visual del sistema. A través del uso de variables nativas personalizadas, Flexbox y CSS Grid, se establece un diseño moderno que mantiene la consistencia gráfica y la adaptabilidad en toda la aplicación. Esta tecnología permite separar completamente la estética de la estructura, asegurando una interfaz de usuario limpia e intuitiva.

> **JavaScript (Vanilla / ES6)**

Funciona como el motor lógico del cliente, implementado sin dependencias de frameworks externos para garantizar un alto rendimiento y ligereza. Aprovechando el estándar ES6 (mediante módulos, importaciones y exportaciones), JavaScript se encarga de interceptar los eventos del usuario, manipular el DOM en tiempo real, gestionar de forma segura el almacenamiento local del navegador (`sessionStorage`) y dirigir la comunicación HTTP asíncrona hacia la API REST del backend.

1.3. Complementos

- **Control de versiones**

> **GitHub**

Se usa como la plataforma central para el control de versiones distribuidas del código fuente y la asignación de tags para empaquetar formalmente las iteraciones del backend y frontend. Facilita la colaboración segura y estructurada mediante el uso de ramas para cada funcionalidad o iteración, además de flujos de trabajo basados en *Pull Requests*. A partir de la versión 2.0 estos PR son obligatorios para la revisión de código por los integrantes del equipo.

Repositorio	Link
Backend	https://github.com/ids-fciencias-7009-2026-2/adopcion-ids-equipo7-api
Frontend	https://github.com/ids-fciencias-7009-2026-2/adopcion-ids-equipo7-frontend

- **Espacio de trabajo**

> **Notion**

Se emplea como el espacio de trabajo colaborativo para la planeación y documentación evolutiva del proyecto. Permite llevar un seguimiento de las tareas, administrar los requerimientos de cada iteración y delegar el trabajo de cada integrante del equipo, facilitando la alineación constante entre los objetivos del desarrollo y la documentación formal entregada.

Sitio: https://www.notion.so/Persona-5-Integrador-DevOps-y-Editor-de-Documento-326de1a623d780db80b1c795bef10bbb

2. Arquitectura General

A partir de la Iteración 2, el sistema completo adopta una **Arquitectura Cliente-Servidor.** Esta decisión separa el proyecto en dos componentes totalmente independientes que se ejecutan en entornos distintos y se comunican a través del protocolo HTTP, utilizando la **API Fetch** nativa de JavaScript:

1. Cuando el usuario realiza una acción (como iniciar sesión o actualizar su perfil), el frontend intercepta el evento, empaqueta los datos en formato JSON y realiza la petición hacia el endpoint correspondiente del backend (API REST). Para operaciones que requieren protección, el frontend recupera el token del almacenamiento local y lo inyecta automáticamente en las cabeceras de la petición.
2. El backend procesa la solicitud y retorna una respuesta HTTP. El frontend interpreta el código de estado devuelto (ej. `200 OK`, `401 Unauthorized` o `409 Conflict`) y actualiza la interfaz en tiempo real para mostrar los nuevos datos o alertar sobre errores al usuario.

2.1. Cliente (Frontend)

El cliente representa la capa de presentación del sistema, ejecutándose directamente en el navegador web del usuario final. En la arquitectura actual, el frontend es una entidad completamente separada de la base de datos y de la lógica de negocio profunda; su rol principal es proveer la interfaz de usuario (UI) capturar los datos ingresados en los formularios y ofrecer mensajes visuales relevantes para el usuario.

Además, el cliente actúa como gestor del "estado" de la sesión del usuario. A través de módulos dedicados, verifica constantemente la existencia de credenciales válidas en el almacenamiento local del navegador (`sessionStorage`) y se encarga de proteger las vistas privadas, expulsando y redirigiendo automáticamente a la pantalla de acceso a cualquier usuario no autenticado.

2.1. Servidor (Backend)

Patrón utilizado: MVC

El backend contiene las principales reglas de negocio y acceso a la base de datos. Para mantener la modularidad y escalabilidad, el backend implementa un patrón MVC adaptado a REST (ideal para API REST) organizado en una la tradicional arquitectura de tres capas: Modelo, Vista, Controlador; aunque esta vez bajo el enfoque de Controller, Service, Repository.

Arquitectura en Capas

- **Controller (capa de entrada)**: Se encarga de recibir y manejar las peticiones HTTP provenientes del frontend, que entran a nuestro sistema por medio de los llamados endpoints, siendo el encargado de llamar a la capa de servicio para procesar la solicitud y en dado caso, devolver la información correspondiente.
- **Service (capa de lógica de negocio)**: Se encarga de la lógica interna ejecutando procesos críticos para llevar a cabo lo requerido por parte de las solicitudes de Controller, generalmente haciéndose aquí los procesos de registro, modificación o autenticación de datos antes de devolver la información a la capa de controllers y/o notificar los cambios a la capa repository, de la cual service extrae la información para llevar a cabo los procesos pertinentes.
- **Repository (capa de persistencia)**: Se encarga de proporcionar la información requerida por las demás capas comunicandose con la base de datos, ejecutando las consultas de inserción, actualización y búsqueda sin alterar la lógica de los datos.

3. Flujo de Autenticación

El sistema implementa un mecanismo de autenticación y manejo de sesiones basado en la generación y validación de tokens únicos. Este modelo garantiza que la comunicación entre el Cliente y el Servidor sea segura y que las vistas privadas estén protegidas contra accesos no autorizados. El ciclo de vida de la autenticación se divide en las siguientes etapas secuenciales:

1. **Inicio de sesión:** El usuario ingresa sus credenciales (correo electrónico y contraseña) en el formulario de la interfaz web. El Cliente captura estos datos y los envía mediante una petición HTTP `POST` al endpoint público de autenticación en el Servidor.
2. **Validación y generación de token:** El Servidor recibe la petición, aplica una función de hash (SHA-256) a la contraseña recibida y la compara con la almacenada en la base de datos. Si las credenciales coinciden, el Servidor genera un Identificador Único Universal (UUID) que actúa como token de sesión, lo asocia a la entidad del usuario en la base de datos y lo devuelve al Cliente como respuesta.
3. **Almacenamiento seguro en el Cliente:** Al recibir una respuesta de autenticación exitosa, el Cliente extrae el token y lo almacena localmente utilizando la API de `sessionStorage` del navegador. Este método de

almacenamiento es volátil y temporal; los datos se destruyen automáticamente si el usuario cierra la pestaña o el navegador, añadiendo una capa de seguridad adicional.

4. **Protección de rutas:** Para acceder a las vistas privadas del sistema, como el panel de inicio (Home) o la vista de edición de perfil, el Cliente ejecuta un script de validación que verifica internamente la existencia del token. Si no se encuentra un token activo en la sesión, el acceso es denegado y el usuario es redirigido de inmediato a la pantalla de inicio de sesión.
5. **Autorización en peticiones (Server-Side):** Cuando el usuario interactúa con funcionalidades protegidas que requieren consultar o modificar la base de datos, el Cliente intercepta la petición HTTP e inyecta dinámicamente el token en la cabecera `Authorization`. El Servidor extrae esta cabecera, busca el token en la base de datos y, si existe y está activo, procesa la solicitud asumiendo la identidad del usuario propietario de dicho token.
6. **Cierre de sesión:** Al seleccionar la acción de cerrar sesión, el Cliente emite una petición `POST` al endpoint de salida. El Servidor invalida el token eliminándolo de la base de datos (estableciéndolo como nulo). Simultáneamente, el Cliente limpia por completo los datos almacenados en el `sessionStorage` y redirige al usuario a la vista pública (Index).

4. Modelo Conceptual de Datos

En la actual iteración, el modelo de datos se centra exclusivamente en la entidad Usuario, la cual es el núcleo para el control de acceso y la seguridad de la plataforma.

Atributos Principales y Propósito

- **Identificador Único (ID):** Cadena alfanumérica (UUID) que garantiza la distinción inequívoca de cada usuario dentro del sistema, evitando colisiones de datos.
- **Nombre:** Almacena la identidad del usuario para personalizar la experiencia en la plataforma y facilitar el contacto humano en el proceso de adopción.
- **Correo Electrónico (Email):** Funciona como el identificador de acceso (login) y el canal de comunicación oficial y seguro entre el adoptante y el publicador.

- **Código Postal (CP):** Atributo estratégico para la visualización geoespacial segura. Permite agrupar las mascotas en un mapa por zonas sin vulnerar la privacidad del usuario al no exponer su dirección exacta.
- **Contraseña (Password):** Credencial indispensable para validar la identidad del usuario y otorgar acceso a las funcionalidades restringidas del sistema.

Consideraciones de Seguridad

Para cumplir con los lineamientos de seguridad y privacidad del proyecto:

- **Cifrado de Credenciales:** Bajo ninguna circunstancia se almacenan contraseñas en texto plano. El sistema utiliza un mecanismo de hashing SHA-256 (procesado en la capa de lógica de negocio) antes de persistir la información en la base de datos.
- **Integridad de Datos:** El correo electrónico se define como un campo único y obligatorio, asegurando que no existan cuentas duplicadas y que siempre exista un medio de contacto válido.
- **Protección de Privacidad:** El modelo está diseñado para que la ubicación solo sea referencial (vía código postal), cumpliendo con el principio de no exposición de información sensible.

Durante el inicio de sesión, el sistema no compara directamente la contraseña en texto plano contra la base de datos. En su lugar, toma la contraseña proporcionada en el request, le aplica el mismo proceso de hash y compara el resultado con el valor almacenado. Esto permite autenticar al usuario sin necesidad de recuperar ni exponer la contraseña original.

Adicionalmente, se realizaron ajustes en las respuestas de los endpoints relacionados con usuarios para evitar exponer información sensible al cliente. Aunque internamente el objeto Usuario contiene el atributo password, en las respuestas HTTP se devuelve una versión sanitizada del objeto, estableciendo dicho atributo en null. Esta decisión se aplicó en operaciones como registro, consulta de usuario y actualización, con el propósito de impedir que la contraseña o incluso su hash sean visibles desde la API.